Memoization Strategies & When NOT to Optimize

5 min read Updated 24 Mar 2026 Topic 28 of 62

5.4 - Memoization Strategies & When NOT to Optimize

🎯 What Interviewers Actually Ask

Question	Level	Frequency
"When should you use React.memo / useMemo / useCallback?"	🟡🔴	★★★★★
"What is the cost of memoization?"	🔴	★★★★☆
"When is memoization harmful?"	🔴	★★★☆☆

The Cost of Memoization

Every memoization tool has a runtime cost. React.memo must shallow-compare all props on every render. useMemo must compare dependencies and store the cached

0.05ms comparing props, potentially *increasing* total render time from 0.1ms to 0.15ms. The optimization only pays off when the render is expensive enough that skipping it saves more time than the comparison costs.

The Decision Flowchart

Before memoizing anything, ask these questions in order:

Is the component actually slow? Profile it with React DevTools. If the render takes under 1ms, don't touch it.

Can you fix it by restructuring? Moving state down or splitting components is free. Try that first.

Is it re-rendering with the same props? If props change on every render anyway, memoization is wasted because the component would re-render regardless.

Is it a leaf component or does it have an expensive subtree? Memoizing a component that renders 50 children is more valuable than memoizing one that renders a single `<p>` tag.

Only after answering all of these should you reach for React.memo, useMemo, or useCallback.

When Memoization Actually Helps

Real scenarios where memoization provides measurable benefit:

1. **A component renders a large list or complex chart**, and its parent re-renders frequently due to unrelated state changes. Wrapping the expensive component in React.memo skips the re-render.

2 **A computation filters or sorts a large dataset**, and the source data doesn't change on every render. useMemo caches the result so sorting 10,000 items happens once, not on every keystroke in an unrelated input.

3 **A callback function is passed to a React.memo child or used as a useEffect dependency.** useCallback stabilizes the reference so the child doesn't re-render or the effect doesn't re-run unnecessarily.

When Memoization Does NOT Help

Scenarios where developers add memoization without benefit:

JSX Copy

```jsx
// ❌ Pointless: adding two numbers costs nothing
const total = useMemo(() => price + tax, [price, tax]);

// ❌ Pointless: no memoized child consumes this function
const handleClick = useCallback(() => setOpen(!open), [open]);

// ❌ Pointless: string concatenation is trivial
const fullName = useMemo(() => `${first} ${last}`, [first, last]);

// ❌ Pointless: React.memo on a component that always receives new props
const Child = React.memo(({ style }) => <div style={style} />);
// If parent creates a new style object every render, memo always re-rende
```

The React Compiler (React 19+)

The React team is building a compiler (previously called "React Forget") that automatically memoizes components and values at build time. When it becomes stable, it will eliminate the need for most manual useMemo, useCallback, and React.memo calls.

This doesn't mean you shouldn't learn these hooks. The compiler is still experimental, and understanding the underlying concepts is essential for interviews and for working with

Common Mistakes

Mistake 1: Memoizing everything "just in case." This adds code complexity and runtime overhead without measurable benefit. Code becomes harder to read and maintain.

Mistake 2: Memoizing the child but not the props. React.memo is useless if you pass a new inline object or function on every render.

Mistake 3: Treating memoization as a substitute for good architecture. If a component re-renders 100 times per second, the fix is usually restructuring the component tree or the state model, not slapping React.memo on everything.

Interview Answer

> "Memoization should be the last tool I reach for, not the first. React.memo, useMemo, and useCallback all have runtime costs. I profile with React DevTools first to find actual bottlenecks. If a component is slow, I try restructuring first: moving state down or splitting the component. If that doesn't help and the component receives the same props across renders, I add React.memo. I use useMemo for genuinely expensive computations on large datasets, and useCallback only when a function is passed to a React.memo child or used as a hook dependency. The React Compiler will eventually automate this, but understanding the principles behind memoization remains important."

Key Takeaways

- Memoization has a **runtime cost** (comparing deps, storing values). It's not free.
- **Profile first.** Don't memoize components that render in under 1ms.

- **Restructure first.** Moving state down and splitting components is free and often more effective.
- React.memo only helps when the component **actually receives the same props** across renders.
- useMemo only helps when the computation is **genuinely expensive**.
- useCallback only helps when paired with **React.memo on the child** or as a **hook dependency**.
- The **React Compiler** (experimental) will automate memoization at build time, reducing the need for manual optimization.

QUICK CHECK — no marks, just for you.

1. A component currently renders in 0.1ms. A developer wraps it in React.memo. According to the lesson, what is the most likely effect on total render time?

- ◯ Total render time is unchanged, because React.memo comparison is free
- ◯ Total render time only changes after the component has rendered at least 100 times
- ◯ Total render time drops to near zero, since React skips the render entirely
- ◯ Total render time may increase, because the prop comparison itself costs time

Check answer

2. A parent renders `<Child style={{ color: 'red' }} />` and Child is wrapped in React.memo. Why does the memoization fail to prevent re-renders?

- ◯ React.memo only works on components that accept zero props
- ◯ The parent creates a brand-new style object on every render, so the shallow prop comparison never matches

- [ ] Style props bypass memoization because React applies them directly to the DOM

Check answer

3. Per the lesson, in which situation does useCallback provide a real, measurable benefit?

- [ ] Whenever a function is defined inside a component, since re-creating functions is expensive
- [ ] When the function is passed to a React.memo child or used as a hook dependency, so a stable reference matters
- [ ] When the function is attached to an onClick handler on a plain DOM element
- [ ] When the function body performs heavy computation, because useCallback caches the return value

Check answer

‹ Previous
Virtualization: Rendering 10,000 Rows

Next ›
Web Vitals, Tree Shaking & Bundle Optimization

LEARN MORE

ARTICLES & DOCS

useMemo — “should you add it everywhere?”
react.dev

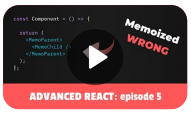
VIDEOS



STOP using React.memo (when not to)
YouTube



When NOT to use React.memo
YouTube



Mastering memoization (advanced)
YouTube

Hand-picked references for this lesson.